Week-5

Aim: Take an example subnet of hosts and obtain a broadcast tree for the subnet.

Source Code:

```
#include <stdio.h>
#include <stdlib.h>
int i, j, a, b, u, v, n, ne = 1;
int min, mincost = 0;
int cost[10][10], parent[10];
/* Function declarations */
int find(int i);
int uni(int i, int j);
int main()
{
   printf("\nImplementation of Kruskal's Algorithm for Broadcast Tree\n");
   /* Read number of vertices */
   printf("\nEnter the number of vertices: ");
   scanf("%d", &n);
   /* Read the cost adjacency matrix */
   printf("\nEnter the cost adjacency matrix:\n");
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         scanf("%d", &cost[i][j]);

         /* Replace 0 with 99 to represent no direct edge */
         if (cost[i][j] == 0)
            cost[i][j] = 99;
      }
   }
```

```
printf("\nThe edges of Minimum Cost Spanning Tree are:\n");
/*
  Kruskal's Algorithm:
  Select the minimum-cost edge at each step.
  Add the edge only if it does not form a cycle.
*/
/* An MST of n vertices contains n-1 edges */
while (ne < n)
{
   min = 99;
   /* Find the edge having the minimum cost */
   for (i = 1; i <= n; i++)
   {
      for (j = 1; j <= n; j++)
      {
         if (cost[i][j] < min)
         {
            min = cost[i][j];

            /* Store the selected edge */
            a = u = i;
            b = v = j;
         }
      }
   }

   /* Find the parent/root of both vertices */
   u = find(u);
   v = find(v);
```

```
        /*
          Check whether adding this edge creates a cycle.
          If u and v are different, the edge can be added.
        */
        if (uni(u, v))
        {
            printf("%d. Edge (%d, %d) = %d\n",
                   ne, a, b, min);

            /* Add the edge cost to total cost */
            mincost = mincost + min;

            /* Increment number of selected edges */
            ne++;
        }
        /*
          Remove the selected edge from further consideration
          by assigning a large value.
        */
        cost[a][b] = cost[b][a] = 99;
    }
    /* Display the total cost of the broadcast tree */
    printf("\nMinimum Cost = %d\n", mincost);

    return 0;
}
/*
  FIND operation:
  Finds the root/parent of a vertex.
```

```
*/
int find(int i)
{
   while (parent[i])
      i = parent[i];

   return i;
}
/*
  UNION operation:
  Joins two different sets.
  Returns 1 if the edge can be added.
  Returns 0 if adding the edge would create a cycle.
*/
int uni(int i, int j)
{
   if (i != j)
   {
      parent[j] = i;
      return 1;
   }

   return 0;
}
```

Output:

```
Implementation of Kruskal's Algorithm for Broadcast Tree

Enter the number of vertices: 6

Enter the cost adjacency matrix:
0 3 1 6 0 0
3 0 5 0 3 0
1 5 0 5 6 4
6 0 5 0 0 2
0 3 6 0 0 6
0 0 4 2 6 0

The edges of Minimum Cost Spanning Tree are:
1. Edge (1, 3) = 1
2. Edge (4, 6) = 2
3. Edge (1, 2) = 3
4. Edge (2, 5) = 3
5. Edge (3, 6) = 4

Minimum Cost = 13
```

Theory:

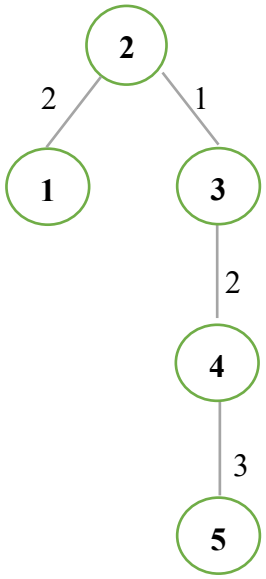
Broadcast Tree

A broadcast tree is a tree structure used to transmit a message from one host to all other hosts in a computer network. It provides a path between the source host and every other host without creating unnecessary loops or duplicate transmissions.

A Minimum Spanning Tree (MST) can be used as a broadcast tree because it connects all the hosts with the minimum total communication cost and contains no cycles. Kruskal's algorithm is one of the algorithms used to construct the Minimum Spanning Tree.

Example

Consider five hosts connected as follows:



The broadcast tree contains these four links and connects all five hosts.

If Host 1 wants to broadcast a message, the message can travel:

Host 1 → Host 2 → Host 3 → Host 4 → Host 5

Thus, every host receives the message through the tree without forming a loop.

Purpose:

The main purpose of a broadcast tree is to provide efficient data transmission from one host to all other hosts while minimizing the total communication cost and avoiding loops.